

Juju-chu

じゅじゅちゅ!



大岡由佳



バージョン管理システム

AI時代に VCS も進化する!

Jujutsu
の特徴



自動 Commit



どこでも Undo



Conflict も履歴の一部

領域展開



じゅじゅちゅ！

jj new で始める Jujutsu × AI ワークフロー

大岡 由佳

くるみ割り書房

※本書に記載されている会社名、製品名などは一般に各社の商標、または登録商標です。

※本書に記載された URL、バージョンなどは予告なく変更される場合があります。

※本書の内容については最大限の努力をもって正確を期していますが、これに基づき生じた運用結果には責任を負いかねますので、ご了承ください。

まえがき

Jujutsu が今、注目を集めています。

バージョン管理システム（VCS）といえば長らく Git 一強の時代が続いており、特に 2010 年代以降に業界に入ったソフトウェアエンジニアの中には、Git しか使ったことがない人も多いでしょう。また GitHub は Git をデファクトスタンダードの地位に押し上げた最大の立役者であり、そのアカウント数は全世界でなんと約 2 億。もはや「GitHub アカウントを持たないエンジニアはエンジニアにあらず」とも言われかねない普及ぶりです。

そんな風潮の中、Git 以外の VCS が話題になるのは非常にめずらしく、「事件」と言ってもいい出来事ではないでしょうか。

なぜ今、Jujutsu なのか？

Jujutsu は 2019 年から開発が始まりましたが、最初の公開リリース（v0.3.0）が出たのは 2022 年のこと。しかし開発コミュニティでしばしば話題に上がるようになったのは、2025 年に入ってからです。折しも AI コーディングエージェントが急速に普及し、「パイプコーディング」なる言葉がパスワードとして広まり出した時期と一致します。

AI エージェントが人間とは比較にならない速度で大量のコードを生成・変更してくれるようになり、コードを書くコストや試行を繰り返すコストが大幅に下がりました。そんな状況と、「人間が 1 行 1 行丁寧にコードを書き、何を記録するかを自分で選び、準備が整ってから慎重に共有する」——そういうワークフローを前提に設計された従来の VCS の間に、齟齬を感じる人が増えてきたのではないのでしょうか。

そして気が付くとそこに Jujutsu という新しい VCS があった。試してみたら AI エージェントによる開発のスピード感にすごくマッチする。この魅力を他の人たちにも伝えねば！ と、昨今の急激な盛り上がりはこのようなして生まれていると筆者は見ています。

「でもそれ、Git でもできるよ」

紹介記事などがバズって Jujutsu が話題になるたびに必ずと言っていいほど見かける反論(?)に、「同じことは Git でもできる」というものがあります。しかし Jujutsu の価値は、「Git にできないことをやる」ことにはありません。

たとえばガラケーと iPhone。両者とも同じく電話ができて、Web ページが見られて、アプリをインストールして使えて、カメラで写真が撮れます。そして iPhone の登場当初には、実際にそのように指摘して iPhone を否定する人たちも少なからずいました。しかし最終的な両者の評価がどうなったかは周知のとおり。できることはたとえ同じであっても、そこで提供されるユーザー体験には決定的な差があったのです。

「できる」と「快適にできる」はちがいます。Jujutsu では Git と同じことをやるにあたっても、それを自動でやってくれたり、操作がおぼえやすかったり、失敗しても簡単にやり直せたりなど、より安心・安全・手軽になっています。

キーワードは「**認知負荷の低さ**」と「**心理的安全性**」。Jujutsu はすべてにおいて、この 2 つが手厚く配慮されているのです。

本書の対象読者

本書が対象として想定する読者像は、以下のようなものです。

- Git の基本的な操作を知っており、日常的に Git および GitHub / GitLab を利用している
- Claude Code や Codex CLI のような AI コーディングエージェントを使っている、もしくは始める予定がある
- Jujutsu は未経験、もしくは使い始めたがまだ初心者の段階

「Git 自体がまだよくわからない人向けの入門書」ではないので、お気をつけください。なお VCS 実演のための教材として React を使っていますが、それが本題ではないため React および Node.js エコシステムについての知識は必須ではありません。適宜、ご自身の主戦場の環境に読み替えてお進みください。

本書を読む上での注意事項

Jujutsu は 2026 年 8 月現在、非常に活発に開発されており、まだ安定版 (v1.0.0) には達していません。実際、プロジェクトの README でも自身を「experimental version control system」と位置づけており、v1.0.0 までは破壊的変更が入る可能性があると言明しています。したがって、本書の記述と最新版の間で各種名称や挙動に差異が生じることがあります。もっとも、本書の内容は単にコマンドの使い方にとどまらず、Jujutsu の設計思想やメンタルモデル、そして AI コーディングへの応用などに及ぶ包括的なものです。今後、表面的な変更があっても、本書の内容は将来にわたって有用だと考えています。

本書の構成

第 1 章では Jujutsu の概要、第 2 章では基本操作、第 3 章で AI との協調開発、第 4 章で各種の応用、第 5 章で FAQ とトラブルシューティングを扱っています。最初から順に読むことをおすすめしますが、まず動かしてみたい人は第 2 章から、AI 連携に強い関心がある人は第 3 章を急いで読んでもよいでしょう。また第 4 章と第 5 章は辞書的にも利用できるようになっています。

なお本書は、シニアエンジニアとジュニアエンジニアの 2 人の対話によるストーリー形式で展開されます。気軽に読んでいただけるよう、また初心者が抱きがちな疑問に逐一答えられるようにこの形式を採用しています。

それでは、これから Jujutsu の世界へご案内いたします。開始の呪文は `jj new`。

本書について

登場人物の紹介

柴崎雪菜（しばさき ゆきな）

都内のとあるインターネットサービスを運営する会社のシニアエンジニア。React の腕を買われて現在の会社にジョインし、テックリードとしてフロントエンド開発チームを一から育てた。新しい技術を見つけてきては見切り発車で導入しようとする傾向があり、チームメンバーからは多少辟易されているが、それらは不思議と後からブレイクすることが多いため黙認されている。

秋谷香苗（あきや かなえ）

柴崎と同じチームに所属する、やる気あふれるジュニアエンジニア。柴崎の一番弟子を自任しており、彼女のやることは何でも真似しようとする。アニメや漫画が大好き。最近では「AI に仕事を奪われたらどうしよう」と不安を抱きつつも、積極的に AI エージェントを開発に利用して、急激な業界の変化に置いていかれないよう努力している。

サンプルコードについて

本文で紹介している各種設定のサンプルコードは、GitHub に用意した下記のリポジトリで公開しています。それぞれのファイルの場所は、対応する本文の脚注をご参照ください。

- ・『じゅじゅちゅ！ jj new で始める Jujutsu × AI ワークフロー』サポート用公開リポジトリ

<https://github.com/klemiwary/Juju-chu-ja>

本文および上記リポジトリに掲載しているコードは、読者の判断でご自由にお使いください。公開文書や動画などに引用するにあたっては、出典を明記していただければ筆者に許可を求める必要はありません。なおこれらのコードをそのまま転載することはご遠慮ください。

正誤表

本文内の記述内容の誤りや誤植についての正誤表は、以下のページに掲載しています。随時更新の予定です。

- ・『じゅじゅちゅ！ jj new で始める Jujutsu x AI ワークフロー』正誤表
<https://github.com/klemiary/Juju-chu-ja/blob/main/errata.md>

本書内で使用している主なソフトウェアのバージョン

- Jujutsu 0.45.1
- jjui 0.10.9
- JJ View 2.7.0
- Claude Code 2.1.260
- Codex CLI 0.153.2

目次

まえがき	3
本書について	6
プロローグ	12
第 1 章 Jujutsu ってどんなツール？	15
1-1. Git は AI コーディングと相性が悪い？	15
冗長な 3 ステートモデル	15
コンテキスト切り替えのコストの高さ	17
履歴の書き換えが複雑で危険	17
コマンドの副作用が大きく取り消しにくい	18
1-2. AI 時代に評価される Jujutsu の特徴	20
作業ディレクトリが即 commit される	23
あらゆる操作が undo 可能	25
Conflict をただの状態として扱う	25
コマンド体系が直交的で、状態依存が少ない	27
コラム：Git はバージョン管理の何を革新したのか	30
第 2 章 使ってみよう Jujutsu	33
2-1. Jujutsu の環境構築	33
2-1-1. Jujutsu のインストール	33
2-1-2. Jujutsu の初期設定	34
2-2. Jujutsu 体験ツアー	36
2-2-1. リポジトリの初期化	36
2-2-2. リポジトリの状態を確認する	38
2-2-3. Change の操作	40
2-2-4. リモートとやりとりする	45
2-3. Jujutsu の 3 種類のログ	48
2-3-1. Revision Log (jj log)	48

2-3-2. Evolution Log (<code>jj evolog</code>)	52
2-3-3. Operation Log (<code>jj operation log</code>)	56
2-4. Jujutsu のメンタルモデル	59
2-4-1. Change とは何か	59
2-4-2. Branch と Bookmark のちがい	60
2-4-3. 匿名 Branch で作業する	62
2-5. よく使う <code>jj</code> コマンド一覧	64
コラム：Jujutsu のルーツ、Mercurial ってどんな VCS ?	66
第3章 実践 Jujutsu × AI ワークフロー	69
3-1. AI エージェントを Jujutsu と協調させる	69
3-1-1. AI エージェントに Jujutsu を使わせる	69
3-1-2. <code>jj</code> コマンドの Permissions 設定	74
3-1-3. Hooks で <code>jj fix</code> を走らせる	77
3-2. 実例で見る Jujutsu × AI による開発プロセス	83
3-2-1. AI が作成した Change の粒度を整える	83
3-2-2. push して PR を作成する	88
3-2-3. Workspace を使った並行開発	94
コラム：Jujutsu に影響を与えたツールたち ①	102
第4章 Jujutsu の高度な術式	105
4-1. ターゲットを指定する冴えたやりかた	105
4-1-1. Revset で Revision をスマートに指定する	105
4-1-2. Fileset でファイルをスマートに指定する	108
4-2. 知っていると便利な裏技的コマンド	110
4-2-1. <code>jj absorb</code>	110
4-2-2. <code>jj arrange</code>	111
4-2-3. <code>jj bookmark advance</code>	113
4-3. Git Hooks の代替戦術	114
4-4. Conflict を半自動的に解決する	118
4-4-1. Mergiraf	118

4-4-2. Weave	121
4-5. Jujutsu の UI ツール	124
4-5-1. jjui	124
4-5-2. JJ View	129
コラム：Jujutsu に影響を与えたツールたち ②	132
第 5 章 Jujutsu 問題解決ガイド	136
5-1. FAQ	136
5-1-1. Git との比較	136
☺ Git にできて Jujutsu にできないことは？	136
☺ merge コマンドはないの？	137
☺ pull コマンドはないの？	138
☺ Git の cherry-pick 相当のことがしたい	139
5-1-2. ニッチな操作・設定について	140
☺ @ を移動させずに、任意の時点でのファイルの内容を確認できる？ ..	140
☺ Change の分割を時系列で行いたい	141
☺ 一時的なログやダンプなど、履歴に含めたくないファイルがある ...	144
☺ Jujutsu のリポジトリ用設定を、そのリポジトリ自身に登録したい ...	144
☺ track 中の bookmark の名前を変更したい	146
5-1-3. Jujutsu のトリビア	146
☺ Jujutsu は Git のラッパーツールなの？	146
☺ Jujutsu の作者ってどんな人？	148
☺ プロダクト名の意味は「呪術」「柔術」どっち？	149
5-2. トラブルシューティング	150
☹ ただの push が知らないうちに force push になっている	150
☹ 「Error: The working copy is stale」という謎のエラーが出る	152
☹ いつのまにか Change に「divergent」という注釈がついていた	153
☹ 画像や動画ファイルを Jujutsu が追跡してくれない	156
☹ PR をマージ後に fetch したら @ が迷子になる	158
☹ まだ作業中のリモートの Bookmark を GitHub 上から削除してしまっ た	159

☺ Claude Code の Permissions 設定で <code>jj log</code> を <code>allow</code> にしていても実行許可 を求められる	159
コラム：JJ ネイティブのホスティングサービスが欲しい！	161
エピローグ	165
著者紹介	167

プロローグ

「ああ——っ、しまった！！」

「急に大きな声出して、いったいどうしたの？」

「Codex にコードを書いてもらってて、前の変更を commit するのを忘れたまま次の指示を出したんです。そしたら意図しない変更で既存のファイルを上書きされちゃって、戻せなくなりました.....」

「AI あるあるな話だね。Claude Code だったら `/rewind` コマンドで戻せるところなんだけど、Codex CLI にはそれに相当する機能がないからね¹」

「最近 Codex のほうが仕事が速くて突破力があるので、こっちを優先的に使ってたんですよ。こんなことなら Claude Code にしとけばよかった.....」

「まあ Claude Code の `/rewind` も万能じゃないけど。対象はエージェントが書き換えたコードのみのため、途中でシェルコマンドでファイルを操作したり、自分が手で編集した差分があったりすると、元通りには戻せない。それに `/rewind` で履歴をいくつもさかのぼって復元しようとするとうと Git の履歴と混ざってしまっで、結果ぐちゃぐちゃになって収集がつかなくなったりする」

「うう、ぴったりあの時点の状態に戻したいというニーズには、どれもきれいに応えてくれないんですね.....。雪菜さんは私よりずっと AI を使いこなしてますけど、そういうやらかしはないんですか？」

「私はもっぱら **Jujutsu** を使ってるから、その手の事故はまずないなあ」

「呪術！？ 雪菜さんって呪術師だったんですか？ しかも時間操作系の呪術を使いこなすとは、1 級呪術師かそれ以上じゃないですか。..... といえば雪菜さんって禪院真希²とキャラ似てますよね」

「いや、それぜったい眼鏡だけで言ってるでしょ。そうじゃなくて私が言ってるの

¹ 2026 年 8 月現在の状況。過去には実験的に `/undo` コマンドを提供していたこともあったが、バグの頻発や VCS と競合する履歴管理がユーザーを混乱させるという懸念から、のちに削除した経緯がある。

² 漫画『呪術廻戦』の登場人物。呪力が一般人並みでしかない代わりに人間離れた身体能力を持ち、さまざまな呪具を使いこなして戦う。肉眼では呪霊が見えないため、ふだんは特殊な眼鏡をかけている。

は『呪術廻戦』³に出てくるような呪術じゃなく、最近注目の新世代バージョン管理システム、Jujutsu のことね」

「へー、そんなツールがあるんですね。私、バージョン管理システムって Git しか知りませんでした」

「最近入ってきた子たちは、たぶん大半がそんな感じだろうね。でも VCS って実は Git 以前にも以後にもたくさんあって、その中で特にいま人気急上昇中なのが Jujutsu なのよ」

「それにしても雪菜さん、いつからその Jujutsu ってのを使ってたんですか？ 同じチームで働いてるのに全然気付きませんでした。……ん？ でも GitHub からは雪菜さんも普通に Git を使ってるように見えたんですけど、いったいどういうことなんですか？」

「ああ、Jujutsu は Git と互換性があって、Git リポジトリをそのままバックエンドストレージとして使えるのよ。だからあえて自分から言わなければ、チームメンバーには Jujutsu を使ってることがわからない。そういえば私が Jujutsu を使い始めたのは半年前くらいだったかな」

「えっ、そんなに前から？ そんな便利そうなものを使ってたのをずっと黙ってたなんてひどいです。私と雪菜さんの仲なのに……」

「ごめんごめん。チーム開発とは直接関係ないツールだし、秋谷さんがそこまで興味を持つと思わなかったから」

「でも今みたいな事故が起きても、Jujutsu を使っていれば取り返しがつくんですよ？ そんなの興味ありまくりですよ！ 私にもその Jujutsu の使い方を教えてください！ 私も 1 級 Jujutsu 師になって時間操作できるようになりたいです」

「そうだねえ、……よしわかった。じゃあ今から時間を作って、秋谷さんに Jujutsu のことを教えてあげよう」

「え、いいんですか？ プロジェクトの進行が遅れちゃいませんか？」

「まあ Jujutsu が使えるようになることで、今後の作業ミスによる時間のロスが激

³『週刊少年ジャンプ』に連載されていた、芥見下々あくだみ げげによる漫画。人間の負の感情から生まれた化け物や呪霊を、呪術によって祓う呪術師たちの戦いを描いたダークファンタジー。コミックスの発行部数は全世界で累計 1.5 億以上。海外でもタイトルはそのまま『Jujutsu Kaisen』であり、日本古流の「柔術 (jujutsu)」と併せて、VCS としての Jujutsu の検索しにくさに貢献している。

減するなら、中長期的に見て圧倒的にプラスでしょう。チームリーダーの職権により、特例として認めます」

「やった！ 雪菜さんによるマンツーマンの Jujutsu レッスンですね。楽しみです！」

第 1 章 Jujutsu ってどんなツール？

1-1. Git は AI コーディングと相性が悪い？

「さっき秋谷さんがやらかしたような事故だけど、あれは秋谷さんが特別に不注意だったから起きたわけじゃない。VCS として使ってる Git の設計が最新の AI コーディングにマッチしていないために、起こるべくして起こったものだと言えるだろうね。まあ20年以上前に設計されたツールなので、AI エージェントがこんなに大量のコードを次々に生成してしまうような事態を想定しろというのは無理な話だけど」

「.....そうか、そうだったんですね。私は悪くない！」

「Jujutsu の説明に入る前に、Git のどこが AI コーディングと相性が悪いかというのを確認しておこうか。そのほうがJujutsuを学びながらその便利さ、使いやすさを実感できると思うから」

冗長な 3 ステートモデル

「Git の最大の特徴であり、同時に初学者が最初につまずきやすいところでもあるんだけど、Git ではファイルの変更が履歴に記録されるまでに次の 3 つの状態を経由するようになってる」

- **Working Tree**（作業ツリー） ファイルを実際に編集する場所。working directory ともいう
- **Staging Area**（ステージング領域） working tree と repository の間にあ
る、commit の準備領域。index ともいう
- **Repository**（リポジトリ） 変更履歴やすべてのファイルが保存されてい
るデータベース

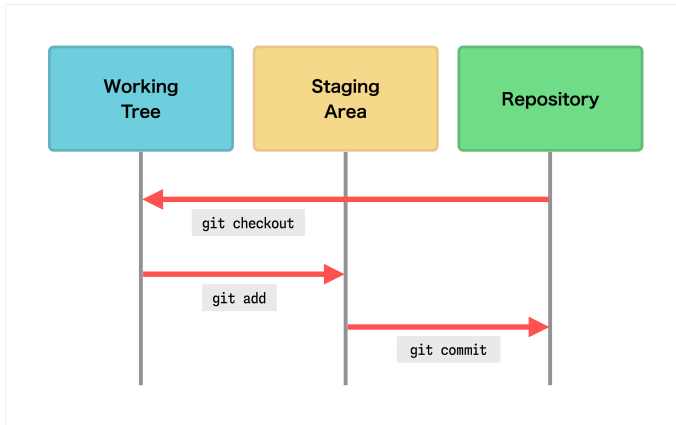


図 1: Git の 3 ステートモデル

「そうそう。最初、なんでわざわざ `git add` して `git commit` する必要があるんだろうと思いましたね。commit だけでいいじゃないですか」

「人間が手作業で開発していた時代には、このモデルは一定の合理性があったんだよ。working tree から変更の一部だけを選んで index して、そこから意味のまとまりごとに commit するという手順を想定してる。staging area は次の commit の下書きに使うためのものなの。これがあることで慎重に作業できて、commit の品質が上げやすかったのね。ほら、Git はもともと Linux カーネルの開発のために作られたものだったから、汚くて意図が読みづらい commit を上げられるとメンテナが迷惑するでしょ」

「ふーむ、なるほど」

「でも AI エージェントが複数ファイルにまたがる数十～数百行のコードを一気に生成してしまう現代においては、このステップが大きな足枷になる。疲れ知らずの AI が次々に書きあげてくるコードを、人間がいちいち `git add` と `git commit` を実行し続けるのは現実的じゃない。その結果、履歴に保存しないまま AI に次の指示を出してしまい、手戻りできなくなったり履歴の海で溺れるという事故が頻発するというわけ」

コンテキスト切り替えのコストの高さ

「実際の開発では、不意に思いついたアイデアを試したくなったり、急なレビュー依頼やバグ修正のタスクが発生することがあるよね。すると working tree を中途半端な状態のまま別の branch に切り替えることを余儀なくされるわけだけど、そういうとき Git では一時退避のために `git stash` を使う」

「はいはい、stash は毎日のようにやってますね。あれ面倒だし、戻ってきたときに stash してたことを忘れてたりすると、悲惨なことになるんですよ」

「割り込みタスクの作業が長引いている最中に、さらに緊急度の高い割り込みタスクが起きたりして stash を複数積み上げたりすると、どれがどの作業の退避データだったかわからなくなるよね。それで復元時に conflict を起こして泣く泣く変更を破棄する、なんて事態もめずらしくない」

「うわ、想像ただけで胃が痛くなりますね……」

「それから Git での作業で、馬鹿にならないのが名前づけのコスト。新しく作業始めるにはまず branch を作り、それに適切な名前を付ける必要がある。commit するにもまず名前が必要で、そして一度決めた名前を思い直して後から変更するのは、できなくはないけど煩雑な手続きが必要になる。ちょっと実験してダメならすぐ破棄するようなものであっても、作業の前に何かしら名前を考えなければならぬのは、認知的な負荷が高い」

「名前づけのコストですか……。あまり意識してなかったですけど、言われてみれば毎回、どんなのつけたらいいか迷ってそこで手が止まっちゃってましたね」

履歴の書き換えが複雑で危険

「人間が慎重にコードを書いていたときと比べて、AI にコードを書かせるコストは劇的に低いので、いったん履歴に保存したものを後から変更したくなる事態は格段に増えてる。ところが Git で履歴を書き換える操作はハードルが高いのよね」

「わかります。私もどうしても履歴編集が必要になったときは、検索したりチャット AI に尋ねたりしながら、恐る恐るやってますもん」

「Git では履歴の書き換えというひとつの概念に対して `git commit --amend`、

第1章 Jujutsu ってどんなツール？

`git rebase -i`、`git reset` といった複数のコマンドがあって、さらにオプションの与え方によって作用範囲がガラッと変わったりするのでメンタルモデルの構築が難しい。これらをすべて暗記してて、場面によって適切に使いこなせる人は少ないと思う」

「ですねー」

「特に `git rebase -i` はエディタ上で commit 一覧を編集するという独特の UI で、ここでの操作ミスが直接履歴に影響してくる。行を消せばその commit が消えるし、`pick` を `drop` に書き換えても消える。エディタを保存して閉じた瞬間に処理が走り始めるので、編集をまちがえたときの絶望感は半端ない。

また途中で conflict が発生すると、commit をひとつずつ順番に適用していく中で何度も conflict の解消を求められることもよくある。その果てに今どの commit を処理中なのか見失って、けっきょく `git rebase --abort` で全部やり直すか、混乱したまま `git rebase --continue` を連発するかのも二択なんてことになりがち」

「そうですね。私はそれが怖いので、後から分割・修正しなくて済むように、未 commit の状態で行けるだけ行くみたいな感じになっちゃうんですね。それでさっきみたいな事故が起きたり、泥だんごのようなまとまりのない巨大な commit で PR を出しちゃったりすることになるんですけど」

コマンドの副作用が大きく取り消しにくい

「Git には、一度実行すると簡単には取り返しがつかない破壊的なコマンドが少なくない。たとえば `git reset --hard`、`git clean -fd`、`git restore` なんかはまだ commit してない変更を容赦なく吹き飛ばす。苦勞して working tree に積み重ねた変更が、たったひとつのコマンドの打ちまちがいでデータ消失につながるのは恐怖以外の何ものでもない」

「以前 Gemini CLI が私の意図とちがう変更をしてくれたので『元に戻して』と言ったら、`git restore` を実行されてそれ以前の作業も全部消されてしまったことがあります。『大変申し訳ありませんでした』って謝られたんですが、後の祭りですね」

「Claude Code なら permissions 設定で特定の Git コマンドの実行を禁止できるけ

ど、設定をうっかり間違えることもあるしね。積み上げた変更を一発で不可逆的に壊すことができる Git を AI にさわせるリスクは高いよ。

また Git のヘビーユーザーには『`git reflog` を使えば操作を戻せるよ』みたいな言う人がいるけど、`reflog` は何でも元通りに戻せるわけじゃない。それに `reflog` の解読は難解だし、安全に元の状態に復元するには Git の内部構造に対する高度な知識が求められる。パニックになっている状態で冷静に `reflog` を操作して、意図通りの状態に戻せるエンジニアはこれまたごく少数だと思う」

まとめ：Git の設計思想と AI 時代におけるミスマッチ

「うーむ、聞いてると Git はまさに人類には早すぎるツールに思えてきました。どうしてこんな仕様になってるんでしょうか？」

「それは Git が元々、『人間が手作業でコードを書き、その変更を整理し、レビュー可能な単位にまとめて共有する』という Linux カーネルの開発モデルを念頭に設計されてるからだね。だから変更を人間同士で慎重に受け渡すワークフローにおいては非常に優れたツールで、長年にわたって存分にその実力を発揮してきた。ただ AI コーディングのシーンでは事情がかなり違ってくる」

- ・ 高速な試行錯誤が起きる
- ・ 大量のコード変更が一気に発生する
- ・ 作業のコンテキストが頻繁に切り替わる
- ・ 後から履歴を整理したい場面が増える

「こういった条件の元では、これまでうまく機能してきたはずの Git の特徴が開発の効率を落とす原因になってしまう。それが AI 時代に Git の使いづらさが際立ってきた理由だね」

「ふむふむ」

「AI コーディングで求められるのは、まず雑に試してその後で安全に整えられるツールなんだよね。そしてまさにこの部分に対して、別の設計思想を持ったバージョン管理システムが注目されるようになってきた」

「なるほど、それが Jujutsu なんですね！」

1-2. AI 時代に評価される Jujutsu の特徴

「雪菜さんを信じないわけじゃないですけど、Jujutsu の人気が急上昇中というのは本当ですか？ これまで私はそんな VCS があるって聞いたことがなかったので」

「まあ2026年中盤の今はまだ、アンテナが高い一部の開発者が熱狂的に支持してるって段階かな。キャズム理論⁴で言えばアーリーアダプタに広まっている段階で、まだキャズムを超えられてない。

何か資料を示してあげたいんだけど、VCS の人気を客観的に計る指標というのはなかなかなくてね。Git 一強時代が長かったので、開発者を対象にしたアンケートでもわざわざ『どんな VCS を使ってますか？』という質問が用意されることもない。あまりフェアとは言えないんだけど、GitHub 公式リポジトリのスター数の遷移を見てみようか」

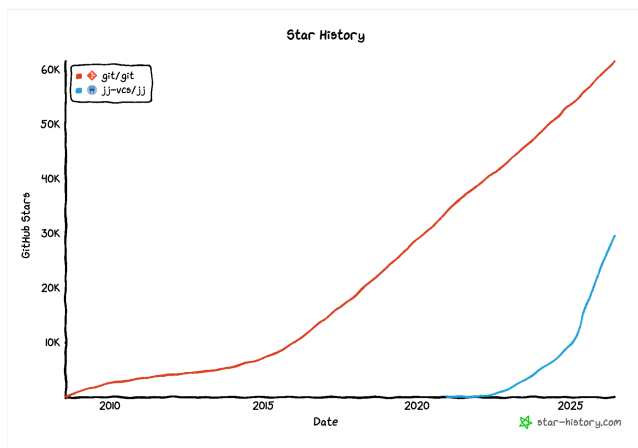


図 2: Git と Jujutsu の GitHub スター数推移 (GitHub Star History, June 2026)

⁴新製品やイノベーションの受け入れを説明する社会学的モデル（テクノロジー採用ライフサイクル）にもとづくマーケティング理論。新製品が普及する過程を5つの顧客層に分け、特に初期市場とメインストリーム市場の間の「深い溝（キャズム）」をいかにして越えるかが最重要の課題とされる。

「おおっ！ Jujutsu のグラフ、ものすごい急カーブの放物線を描いてますね。特に 2025 年中盤からどーんとブーストがかかってる感じ。このまま行けば、Git を抜く日も遠くなさそうじゃないですか。でも『フェアじゃない』っていうのは？」
「Git は GitHub 上で開発しているわけじゃなくて、ミラーリポジトリを置いてるだけなんだよね。公式の本体は Linux カーネルと同じく kernel.org でホスティングされてるの⁵。いっぽう Jujutsu は GitHub リポジトリが公式となっている」

「でもまあ、一般ユーザーにとっては GitHub がすべてみたいなものですから、人気のバロメーターとしては信頼できるんじゃないですか。Jujutsu が急激に人気を集めているというのはわかりました。では、この人気の要因は何なのでしょう？」

「秋谷さん、さっき 2025 年中盤からブーストがかかってるって言ったよね。その時期に何があったかおぼえてる？」

「..... うーん、何でしたっけ？」

「5 月に Claude Code が正式にリリースされたじゃない。それまでのコーディング AI ツールは IDE の形で提供される、サジェストとチャットによる簡単なコード変更が主体のものだった。Claude Code はターミナル上で動作するエージェント型コーディング AI ツールで、指示を与えれば自律的に高品質なコードを一気に生成してくれる。

Claude Code の登場によって、人間が書くコードより AI が書くコードのほうが圧倒的に多いという状況が生まれた。そして開発者の人気を集める Claude Code に遅れまじと Codex CLI や Gemini CLI が追随、Cursor がエージェント型に転換したりと、エージェントによる AI コーディングのトレンドが決定的になったよね」

「そうでしたね。もう遠い昔のように思えますけど、つい最近まですべてのコードを自分で手書きしてたんですよ」

「Jujutsu の人気は、Claude Code や Codex のような AI コーディングエージェントの普及に引っぱられる形で伸びてきているように見える。私もこの時期に立て続けに書かれた AI コーディングと絡めて紹介する記事を読んだことで、Jujutsu を始めてみようと思ったの。反響が大きかった記事にはこんなものがある」

⁵ <https://git.kernel.org/pub/scm/git/git.git>

- Use Jujutsu, Not Git: Why Your Next Coding Agent Should Use Jujutsu⁶
Git ではなく *Jujutsu* を使おう：次世代コーディングエージェントが *Jujutsu* を使うべき理由
- Avoid Losing Work with Jujutsu (jj) for AI Coding Agents⁷
AI コーディングエージェント向けの *Jujutsu* で作業が消えないようにする
- Why Jujutsu (jj) Is Perfect for AI-Generated Code⁸
Jujutsu が AI 生成のコードに最適な理由
- Towards an AI-Native Development Workflow (Using Jujutsu as the Backbone)⁹
AI ネイティブ開発ワークフローの構築に向けて (*Jujutsu* を基盤に)

「へーえ。識者にこれだけ *Jujutsu* は AI と相性がいいと言われたら、その気になっちゃいますね。でも不思議なんですけど、AI コーディングが普及する前から *Jujutsu* ってあったんですよね？ それを想定して設計してたわけじゃないでしょうに、なぜ AI コーディングにフィットしてたんでしょうか？」

「*Jujutsu* の開発が始まったのは 2019 年で、その時点ではほとんどの人が AI が自律的にコーディングしてくれる未来を予想しえなかったから、もちろんそれを見据えて作られたわけじゃない。作者の Martin von Zweigbergk 氏の目的は『人間の認知負荷を下げる』と『Git の複雑なメンタルモデルの解消』にあったんだそうだよ¹⁰。

AI エージェントというのは、ミスを恐れず試行錯誤の高速ループを回す存在だから、それと共働するのは人間にとってものすごく認知負荷が高い。だから人間の認知負荷を下げることを追求した *Jujutsu* の設計が、その帰結としてこの状況にマッチしたと言えるだろうね」

「ふーむ、なるほど」

⁶ <https://slavakurilyak.com/posts/use-jujutsu-not-git>

⁷ <https://www.panozaj.com/blog/2025/11/22/avoid-losing-work-with-jujutsu-jj-for-ai-coding-agents/>

⁸ <https://cesar.velandia.co/why-jujutsu-jj-is-perfect-for-ai-generated-code/>

⁹ <https://ianbull.com/posts/jj-vibes>

¹⁰ 「Solving Git's Pain Points with Jujutsu (with Martin von Zweigbergk) - YouTube」
https://www.youtube.com/watch?v=ulJ_Pw8qqqE

「では、その AI 時代になって評価されてる Jujutsu の特徴をひとつずつ説明してこう」

作業ディレクトリが即 **commit** される

「Jujutsu は Git とちがって staging area を持たない。状態は Git の working tree に相当する **Working copy** および **Repository** の 2 つのみ。そしてこれが Jujutsu の最大の特徴なんだけど、working copy におけるファイルの変更は自動的に commit される。ユーザーが `git commit` のようなコマンドを実行しなくても working copy の状態がスナップショットされ、それが repository に反映されるの」

「えっ。add 操作が必要なくて、さらに commit も勝手にしてくれるんですか。それはいったいどういうからくりで実現されてるんでしょう？ デーモン¹¹がファイルの状態を監視してるとか？」

「いや、Jujutsu のコマンド体系は `jj log` や `jj diff` のように `jj` がメインコマンドなんだけど、この `jj` コマンドが実行されるタイミングでスナップショットが作成されるようになってる。そして直前のスナップショットと差分があれば commit されるというしくみになってる」

¹¹UNIX や Linux などの OS において、バックグラウンドで常時稼働して Web サーバやネットワーク管理など特定のタスクを自動的に処理するプログラムのこと。

第1章 Jujutsu ってどんなツール？

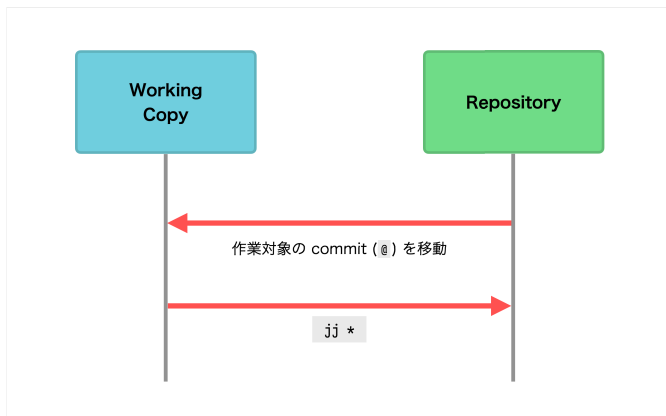


図 3: Jujutsu の 2 ステートモデル

「へーえ。add し忘れ、commit し忘れが原因のミスがなくなるのはうれしいですね。でも commit のタイミングを自分でコントロールできないとなると、履歴がカオスなことになっちゃうんじゃないでしょうか？」

「後で実際に体験してもらうときにくわしく説明するけど、Jujutsu には commit を包括する単位概念として **Change** というものがある。今の時点でもわかるようざっくり表現すると、change は複数の commit を時系列に格納するコンテナみたいなものね。通常、Jujutsu の履歴は change 単位で表示されて、その中の commit 群を見るには余分な操作が必要になる。名前を付けたり、粒度を調節したり、順番を入れ替えたりするのはこの change が基本的な単位なので、勝手に commit が積み上がって履歴が汚くなるという心配をする必要がないのよ」

「ふーむ、そうなんですか」

「公式ドキュメントではこの特徴を『**Working-Copy-as-a-Commit**』という言葉で表現してる。Jujutsu はシンプルな 2 ステートモデルとこの特徴によって `git add` や `git commit`、`git stash` に相当する操作を必要としない。だから従来の VCS と比べて、Jujutsu ではファイルの状態管理にまつわる認知負荷が格段に低いの」

あらゆる操作が undo 可能

「ふだん Git を使っていて『今の操作、undo したい!』と強烈に思うことはない?」
「それはもう、毎日のようにありますね。むしろなんでエディタのように undo 機能がないんだと常日ごろから怒ってます」

「Jujutsu にはその **ユニバーサル Undo** があるの。リモートが絡むものはさすがに完全に元通りにはできないけど、ローカルでのあらゆる操作が undo / redo できる」

「それはすごい! もうそれだけで Jujutsu 使いたいわって思っちゃいますね」

「Jujutsu には通常の commit のログの他に **Operation Log** (操作ログ) というものがある。これは `jj` コマンドの実行履歴で、working copy のスナップショットと結びついてる。だから単純な undo だけでなく、履歴をさかのぼって任意のコマンドが実行されたときの状態に戻すのも、その操作がなかったことにするのも自由にできるわけ」

「Git ではいつも rebase とかの履歴操作系コマンドはおっかなびっくり実行して maisけど、undo があるなら『とりあえずやってみて、ダメなら戻してやり直す』アプローチがとれて安心ですね」

「そう、取り返しがつかなくなるかもしれない恐怖から解放されることで、試行錯誤のハードルが劇的に下がるんだよね。Git だと履歴操作系のコマンドを AI エージェントに実行させるのは怖くてなかなかできないけど、Jujutsu ならそれも問題なくできる。この圧倒的な心理的安全性の高さは、一度経験したら手放せなくなるよ」

Conflict をただの状態として扱う

「Git で merge や rebase、cherry-pick するときに怖いのが conflict。Git で conflict が発生すると他の操作が厳格にブロックされて、解決するまで強制的に作業がストップしてしまう」

「怖いですよね。PR を出す前に最新の main branch を取り込むとき、いつも『conflict が起きませんように』と祈ってます」

「いっぽう Jujutsu では **conflict はただの状態** として扱われ、発生してもそのまま

第1章 Jujutsu ってどんなツール？

commit されて操作を一切ブロックしない。たとえば rebase 中に conflict が発生したときでも、conflict 状態を抱えたまま rebase は完了する。この特徴は公式ドキュメントでは『**First-class Conflicts** (一級市民としての conflict)』と呼ばれてる」

「えっ、それって嬉しいんですか？ conflict ってすぐ解決しないといけないものじゃないの？ だって残ってると気持ち悪いじゃないですか」

「『conflict はすぐ解決しないといけないもの』というのは、Git 脳な人の思い込みだね。Git で conflict を解決しないと先に進めない『例外』として扱ってるのは、技術的な必然じゃなく実装上の制約からだもの。じゃあこの First-class conflicts が嬉しいと感じる具体的なケースを挙げてみようか」

- ある branch を rebase 中に発生した conflict を解決するには、その実装意図を知っている A さんに確認する必要があるが、その当人は休暇中で明日まで戻らない。Git なら翌日まで rebase をあきらめるところだが、Jujutsu ならとりあえず rebase して解決を後回し、もしくは雑に手直した状態で作業を進められる
- merge 中に発生した複数の conflict の解決を AI に任せたい。Git だとすべての conflict を解決しないと commit できないため、まちがっていたときに特定の conflict だけ復元して検証し直すようなことが難しい。Jujutsu ならピンポイントで復元させ、何度でも解消作業をやり直すことができる

「ああ、なるほど。そう言われてみれば便利かもしれませんね。AI に conflict を解消させるなんて Git では考えられませんでしたけど、ひとつずつ解消させてダメなら undo でやり直すことができるし、怯まず任せられそう」

「加えて Jujutsu では過去の commit を編集すると、そのファイルの変更が自動的に子孫の commit にも伝播される。だから conflict を解決するにしても、それが発生してる commit に移動してそのファイルを編集するだけで、あらためて自分で rebase する必要がない。後回しの解決も最小限の労力でできることがわかってるから、心理的な負担も小さいよ」

コマンド体系が直交的で、状態依存が少ない

「Jujutsu を使っていて感じるのは、コマンドがすごく覚えやすいということ。まず対象が履歴そのもののコマンドは、やりたいこととの 1 対 1 の動詞になってる。たとえば履歴の破棄は `jj abandon`、分割は `jj split`、集約は `jj squash` のようにね。 `git checkout` や `git reset` のようにオプションや引数によって操作が根本的に変わるようなコマンドがなく、人間がやりたい変更操作がそのままコマンドに対応してるから、『この操作はどのコマンドだったかな』のように迷うことがない」

「ふむふむ、多義的コマンドが少ないのは嬉しいですね」

「対象が履歴そのものではないコマンドの場合も、論理的な階層に整理されててわかりやすい。たとえば Git で branch 操作は新規作成が `git branch <branch-name>` か `git checkout -b <branch-name>` や `git switch -c <branch-name>`、一覧表示が `git branch`、削除が `git branch -d <branch-name>` となっていて一貫性があるとはいえない。Jujutsu ではそれが `jj bookmark create`、`jj bookmark list`、`jj bookmark delete` と、『何を対象に、何をするか』がコマンドの並びからわかるようになってる」

「たしかに Git の branch コマンド、どれがどの操作だったかすぐわからなくなって、毎回検索してました。でも Jujutsu のコマンドがそういう作法になってるなら、私でも覚えられそう」

「さらに Jujutsu では、基本的なオプションや対象の指定作法が多くのコマンドで共通してる。オプションによる履歴の指定は `-r/--revision` だし、`-f/--from` や `-t/--into`、`-o/--onto` などは履歴操作系のコマンドにおいて同じ意味で使われている。だからひとつ覚えた知識を別のコマンドにも応用しやすいのね」

「なるほど。Git だとコマンドごとに別々の呪文を覚えさせられる感じがありますもんね」

「ソフトウェア設計の世界では、ひとつの要素について知れば他の要素でも類推が効くような性質のことを『直交性が高い』と表現するんだけど、Jujutsu のコマンド体系はまさに直交性が高いと言えるだろうね。だから何回か使っているうちに、『この操作ならたぶんこう書けるはず』と自然に手が動くようになる」

「直交性という言葉は初めて聞きましたけど、つまりはシンプルで直感的ってこ

とですね」

「加えて、Jujutsu のコマンドは状態依存が少ないというのも挙げておきたい。Git のコマンドは常に HEAD¹² が起点であり、working tree と staging area の状態にも影響されるので、同じコマンドを実行してもシーンによって結果が大きく異なってくる。いっぽう Jujutsu では staging area がなく変更が自動的に commit されるため、working copy は履歴グラフ上の数ある地点のひとつでしかない。そしてコマンドの多くが引数として履歴の ID を受け取れるため、working copy がどこを指している、ツリーグラフ上の履歴同士を直接操作できるの」

「うーん、たとえば？」

「`jj squash` は履歴を集約するコマンドだけど、`jj squash -f <from-id> -t <to-id>` のように集約元と集約先を明示的に指定すれば、working copy がグラフ上のどこにあっても結果は同じになる。Git で同じことをしようと思ったら、その前に目的の場所に `checkout` する必要があるよね」

「ああ、たしかに。でも Jujutsu のようにターゲットを毎回オプションで指定するやり方も、キーストロークが多くなって面倒というデメリットがあるように思えますけど？」

「だから多くのコマンドでは、オプションが省略された際のデフォルト値が用意されてるの。`jj squash` を引数なしで実行すると、working copy がその親の履歴に集約される。また `-f` だけを指定したり `-t` だけを指定することもできるよ」

「なるほど。ちなみにコマンドがの状態依存が少ないと、何が嬉しいんでしょうか？」

「working copy をいちいち移動させなくても、やりたい操作がどこからでもできるよね。また操作前に現在の状態を正確に把握しなければいけない認知負荷から解放されるし、ターゲットを明確に指定することは操作ミスを減らすことにもつながる。コマンドが直交的で状態依存が少ないというのは Jujutsu に慣れてしまうと当たり前すぎて無意識になってしまうけど、たまに Git に戻って操作することがあると、いかに Jujutsu が便利だったのかを実感させられるね」

¹² Gitにおいて、現在自分が作業している場所を指し示すポイントのこと。通常は現在の branch の最新 commit になる。

まとめ：Jujutsu は雑に始めて、後から整えることができる VCS

「ここまで挙げてきた特徴を総合すると、Jujutsu の根底にある思想が見えてくる。それは『とりあえず雑に始めて試行錯誤し、体裁は後から整えればいい』というスタイルね」

- working copy が自動的に随時 commit されるため、変更したファイルの記録状態を気にする必要がない
- 最初から履歴単位に完璧な粒度を目指したり、適切な名前を考えたりしないでいい。必要になった時点で分割・集約・入れ替えができ、実情に合わせて名前を付け替えられる
- あらゆる操作が undo できるので、ミスを恐れずに果敢に試行錯誤を重ねることができる
- conflict はすべてをブロックする例外ではなく、commit が保持する状態。解決は後回しにして作業が続けられる
- コマンド体系が直交的で状態依存が少ないため、思いついたときにすぐその場で履歴を整えられる

「なるほど……。何ごとも先にきっちり決めておかなきゃいけなくて、後から変更するには面倒な手続きが必要になる Git とは対照的です」

「この Jujutsu のスタイルは、人間は失敗するものであることを前提に、小さな単位で修正を繰り返しながら完成度を高めていくアジャイル開発に通底するところがあるよね」

「あー、たしかに」

「AI エージェントは人間と同じかそれ以上に失敗するし、人間よりも圧倒的に高速に試行錯誤を繰り返して大量のコードを生成する。Jujutsu の AI エージェントに対する親和性は偶然の一致ではなく、人間の脳に優しい設計を突き詰めた結果の必然的な帰結と言えるだろうね」

「人間にも AI にも優しい Jujutsu……。がぜん興味が湧いてきました。早くさわってみたいです！」

■ コラム：Git はバージョン管理の何を革新したのか

第1章はGitの設計が時代に合わなくなっているという観点から論を進めましたが、GitがVCSの歴史の中で果たした大きな役割についても述べておかないとフェアではないかもしれません。Jujutsu 開発の動機のひとつはGitのつらみを解消することにあったわけですが、それは裏を返せばJujutsuはGitという偉大な先人の肩の上に作られたプロダクトだということでもあるからです。

Gitが現場に普及しだしたのは2010年代前半からでしたが、それ以前はVCSといえば集中管理型のプロダクトが一般的で、中でもSubversion¹³が広く使われていました。単一の中央リポジトリを皆で共有するためすべての操作にサーバ接続が必要で、commitはもちろんログやdiffを見るのすらネットワーク越し。オフラインで何かまとまった作業をするというのが基本的に難しく、commitすると即座に他メンバーに共有されるため、中途半端な状態で細かく履歴を刻むことができません。だから手元でたくさんの変更を重ねてリリースできる状態になったらようやくひとつcommitするとか、1日の作業の終わりにまとめてcommitするなどという、およそVCSの意味が半減するような運用も少なくありませんでした。

それがGitの普及により状況が一変します。履歴のほぼ完全なコピーが常にユーザーの手元に存在し、日常的な操作の多くがローカルで完結する。commitは自分のローカルリポジトリに対して行うものなので、自分の裁量により好きな粒度で履歴を刻むことができる。たとえサーバが落ちたりオフラインの環境だったりしても、作業を続けることが可能。

Gitでは中央リポジトリは概念上はメンバー共有のための合流地点のひとつにすぎず、各メンバーそれぞれがほぼ完全な履歴を備えたりリポジトリを保持することになります。もし中央リポジトリが破損した場合でも、メンバーのリポジトリから復元できる。Subversionではメンバーの手元のファイルは中央

¹³<https://subversion.apache.org/>

リポジトリに従属するものでしたが、Git ではメンバーそれぞれが中央と対等なリポジトリを持つ。これは開発における権力構造やメンバーの意識を変えるものでもありました。

またGitは他のVCSと比べて各種操作が飛び抜けて速かった。ほとんどの操作がローカルで完結するというのもありますが、設計ターゲットがLinuxカーネルだったということもあり、設計段階からネットワーク転送量の少なさやログ閲覧、mergeの速度などに性能要件を立てていたというのも大きい。従来のVCSではファイルの初期状態から変更された差分を順次記録していく方式が取られていたのに対し、Gitではcommit時にファイルシステム全体のスナップショットを記録します。そのため特定のバージョンを取り出すには、直接該当するcommitのポインタを読みに行くだけなので、非常に高速に動作します。

さらにGitはbranchの意義も変えました。Subversionのbranchは単にプロジェクト全体のコピーが別ディレクトリに作られるというものだったため、その操作は心理的に重いものでした。加えてデータモデル的にmergeの追跡が弱く、本来不要なconflictが起きやすかったために、それを嫌ってbranchをあえて使わない現場も多かったのです。いっぽうGitでは履歴そのものがDAG¹⁴として構造化されており、branchは特定のcommitを指す単なるポインタという非常にスマートな設計となっています¹⁵。Gitのbranchは動作が軽く、merge時のconflictも少ない、ローカルのみで作成することも可能ということが、その心理的コストを大きく下げました。小さな機能ごとにfeature branchを切ったり、複数のbranchで作業を並行して進めたり、個人レベルの実験場としてローカルbranchを使ったりといった開発スタイルは、Gitの普及により初めて可能になったことでした。

¹⁴Directed Acyclic Graphの略で、日本語では「有向非巡回グラフ」。矢印（有向）で結ばれたノードが、決して閉路（循環）を形成しないグラフ構造のこと。

¹⁵「2-4-2. BranchとBookmarkのちがひ」で説明します。

Git によって VCS が会社や組織の管理下において「使わなければならない」ものから、個人の生産性を上げるために「自ら使いこなしたい」ものへと位置づけが変わったと言えるでしょう。Git の登場は VCS そのものを民主化し、開発者ひとりひとりの手に平等に履歴とそのパワーを持たせてくれたというのが筆者の認識です。

第2章 使ってみよう Jujutsu

2-1. Jujutsu の環境構築

2-1-1. Jujutsu のインストール

「ここからは Jujutsu を実際に使いながら学んでいってもらいたいんだけど、何はともあれまずはインストールだね。Jujutsu は Rust で開発されていることもあって、公式ドキュメントでは Cargo による手順が優先的に書かれてるんだけど、Rust 大好きユーザーでもない限りこれは避けたほうがいい。ビルドに時間がかかるし、アップデートの管理が煩雑になるからね」

「はい。当面 Rust をさわる予定はないので、このためだけに Rust 環境の構築から始めるのは遠慮したいです」

「幸い私も秋谷さんも Mac を使ってる。だから Homebrew を使えば以下のように簡単にインストールできるよ」

```
$ brew install jj
```

「それからこれは必須じゃないんだけど、シェルに設定を書いておくとコマンド実行時に便利だよ。自分が使ってるシェルに応じて、設定ファイルに次の1行を追記しておこう」

```
# zshの場合
source <(COMPLETE=zsh jj)

# fishの場合
COMPLETE=fish jj | source
```

「これは何をやってるんですか？」

「`COMPLETE=zsh` と環境変数をセットした状態で `jj` を実行すると、zsh 用のシェル補完スクリプトが出力されるの。zsh 起動時にこれを読み込ませることで、`jj` コマンドの最新の補完定義が動的に生成・適用されるようになる。サブコマンドや

オプションを途中まで書いて Tab キーを押すと候補一覧を表示してくれたり、コマンド入力を補完してくれたりするの」

「へー、便利そうですね。私も設定に入れとこうっと。ところで自宅には Windows マシンもあるんですけど、Windows でのインストールはどうしたらいいんでしょうか？」

「WSL¹⁶ 環境なら Mac と同じく Homebrew を使うのがいいだろうね。ネイティブ環境なら WinGet に `jj-vcs.jj` という名前でもパッケージが登録されてるので、それをインストールする。ただし改行コードやシンボリックリンクの取り扱いに注意点があるので、公式ドキュメントの該当ページ¹⁷をよく読んでおいてね」

「わかりました」

「それから Jujutsu とは直接関係ないけど、GitHub CLI (`gh`)¹⁸ をもしまだ入れてなかったら、これを機にインストールしておこう。`jj` に加えて `gh` コマンドを AI エージェントに実行させることで、ワークフローがかなり効率的になるよ」

2-1-2. Jujutsu の初期設定

「Jujutsu には設定可能な項目が膨大にあるけど、最初に必要なものはごく一部だけ。まずはユーザー名とメールアドレスを登録しておこう」

```
$ jj config set --user user.name "kanaetti"
$ jj config set --user user.email "kanaetti@skydome.co.jp"
```

「`jj config set`¹⁹ は設定ファイルに設定値を書き込むコマンド。`--user` ならユーザーレベルの、`--repo` ならそのリポジトリ限定の設定がターゲットになる」

¹⁶Windows Subsystem for Linux のこと。Microsoft が提供する、Linux のバイナリ実行ファイルを Windows 上でネイティブ実行するための互換レイヤー。仮想マシン上で本物の Linux カーネルが動作し、Ubuntu や Debian といったディストリビューションをインストールして使うことができる。

¹⁷「Working on Windows - Jujutsu docs」<https://docs.jj-vcs.dev/latest/windows/>

¹⁸<https://cli.github.com/>

¹⁹「`jj config set` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-config-set>

「これ、設定をしないまま使い始めるとどうなるんですか？」

「履歴の author 情報が欠落してしまい、そのままだとリモートに push できない。だからあらためてユーザー情報を設定したうえで、`jj metaedit --update-author <TARGET>`²⁰ を実行して履歴の author 情報を書き換える必要がある」

「面倒ですね。新規インストール時には忘れないようにしないと」

「それからエディタの設定も変更しておこう。そのままだと必要な際に Jujutsu によって起動されるエディタが、nano という馴染みのないものになってしまう²¹」

```
# Vimの場合
$ jj config set --user ui.editor "vim"

# VS Codeの場合
$ jj config set --user ui.editor "code --wait"
```

「VS Code の起動で渡してる `--wait` オプションって何ですか？」

「Jujutsu がエディタを起動する際、その終了を検知して編集内容を受けとる必要がある。GUI エディタをターミナルから開くと通常は別プロセスで起動されて、編集内容が宙に浮いてしまう。`--wait` をつけると VS Code は開いたタブが閉じられるまでそのプロセスをブロックし続けるので、Jujutsu がその完了を認識して編集内容を正しく受け取れるようになるの」

「ふむふむ、なるほど」

「なおターミナルから `code` で VS Code が起動できない場合は、`* + shift + p` でコマンドパレットを開いて『shell』を入力、表示された中から『Shell Command: Install 'code' command in PATH』を実行すると起動できるようになるよ。

ではこの状態で `jj config edit --user`²² を実行してみよう。さっき指定したエディタで次のような内容が表示されるはず」

²⁰ 「`jj metaedit` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-metaedit>

²¹ Mac および Ubuntu 系 Linux で環境変数 `$EDITOR` が未設定時の挙動。Windows の場合はメモ帳 (Notepad) が起動する。

²² 「`jj config edit` - CLI reference - Jujutsu docs」
<https://www.jj-vcs.dev/latest/cli-reference/#jj-config-edit>

```
#:schema https://docs.jj-vcs.dev/latest/config-schema.json

[user]
name = "kanaetti"
email = "kanaetti@skydome.co.jp"

[ui]
editor = "vim"
```

「それからこのファイルはどこにあるのか。それも `jj config` で知ることができる」

```
$ jj config path --user
/Users/kanaetti/.config/jj/config.toml
```

「このファイルを直接編集することでも、その設定が反映されるんですよね？」
「もちろん。だからこのファイルを Dropbox の共有フォルダなどに置いて、そこからシンボリックリンクを張れば、複数マシンで Jujutsu のユーザー設定を共有できたりするよ」

2-2. Jujutsu 体験ツアー

2-2-1. リポジトリの初期化

「じゃあ Jujutsu を使ってファイルのバージョン管理をやっていこう。React アプリを教材にするのがいいかな。ランタイムに Node.js、パッケージ管理に pnpm を利用するので、入れてない場合は先に準備しておく。そしてこれらのインストールには `mise`²³ を使う」

```
$ brew install mise
$ mise use -g node pnpm
```

²³ プログラミング言語のランタイムや各種 CLI ツールを、複数バージョンを管理しつつインストールできるパッケージマネージャ。タスクランナーと環境変数の設定機能を併せ持つ。
<https://mise.jdx.dev/>

「じゃあ React プロジェクトを作成して、そこを初期化。Jujutsu の管理下に置くよ」

```
$ pnpm create vite jj-tour
◇ Select a framework:
| React
◇ Select a variant:
| TypeScript
◇ Which linter to use?
| ESLint
◇ Install with pnpm and start now?
| No
◇ Scaffolding project in /Users/kanaetti/projects/jj-tour...
└ Done.

$ cd jj-tour/
$ pnpm install
$ jj git init
Initialized repo in "."

$ ls -d .*
.git/      .jj/      .gitignore
```

「初期化のコマンドって `jj init` じゃなくて `jj git init`²⁴ なんですか？」

「Jujutsu はプラグイン可能なストレージアーキテクチャを採用していて、Git をバックエンドのストレージとして使うことができる。だからこのコマンドは Jujutsu を有効にした Git リポジトリを作成するものね。古い記事では Jujutsu リポジトリと Git リポジトリの共存を有効にするために `--colocate` オプションが必要とされていることがあるけど、今ではデフォルトで指定されるので不要」

「すでに Git で管理してるリポジトリで、後から Jujutsu を使いたい場合はどうしたらいいんでしょう？」

「同じく `jj git init` で OK。Git で作成した過去の commit 履歴は、Jujutsu の履歴としてそのまま取り込まれるよ」

²⁴ 「`jj git init` - CLI reference - Jujutsu docs」

<https://www.jj-vcs.dev/latest/cli-reference/#jj-git-init>

第3章 実践 Jujutsu × AI ワークフロー

3-1. AI エージェントを Jujutsu と協調させる

3-1-1. AI エージェントに Jujutsu を使わせる

「私は最近 Git の操作も AI に任せるようになって、『じゃあここまでのところをcommitしてpushして』みたいによく頼んでるんですね。Jujutsu の操作を AI に任せることってできるんですか？」

「もちろんできるよ。Jujutsu なら気軽かつ安全に履歴を巻き戻せるので、むしろ Git より思い切ってバージョン管理を任せられる。それにstagingがないしcommitも自動的に行われるから、具体的にバージョン管理の操作を指示しなくても、タスクを依頼するだけで適切な単位に履歴が整理されるようにもできる」

「それはいいですね。ちなみに雪菜さんはずっと AI コーディングと併せて Jujutsu を使ってるんですね。AI エージェントはデフォルトでは Git しか使ってくれないと思うんですけど、Jujutsu を使わせるのにどんな工夫をしてるんですか？ そのあたりの知見はめっちゃ貴重だと思うので、ぜひ共有してくれると嬉しいです！」

「わかった。ただ AI エージェントごとに機能や特性がけっこうちがうし、目まぐるしく新機能が追加されている最中なので、ベストプラクティスはなかなか定まらない。コーディングエージェントの中では Claude Code が最大シェアで私もヘビーユーザーなので、その 2026 年 8 月時点での最適と思われる方法を中心に説明しよう。あと Codex CLI についてもその都度、補足的に触れておこうか」

「お願いします。私も日常的に使ってるのはほぼその 2 つなので」

「じゃあまずは、AI エージェントに VCS として Git ではなく Jujutsu を使ってもらうための指針を示す方法からね。実は Claude Code や Codex の最新モデルはすでに Jujutsu の基本的な概念やコマンドを理解していて、`CLAUDE.md` や `AGENTS.md` に『Git ではなく Jujutsu を使って』と書くだけである程度は動くのね」

「え、そうなんですか？」

「ただ依然としてコーディングタスクの学習の大半は Git の環境で行われているので、Jujutsu ネイティブなワークフローが身についているわけじゃない。だからさっきのような簡単な指示だけだと次のようなことが起こりうる」

- `git` コマンドを、近似する `jj` コマンドに単純に置き換えただけの操作を行う
- `working-copy commit` の状況を確認することなくタスクを開始して、履歴が混ざってしまう
- `bookmark` を、常に最新の `commit` に移動する Git の `branch` と同様に考える
- `push` の際に `bookmark` を適切に扱えない
- `conflict` が発生すると、そこで処理を中断させてしまう
- Jujutsu 固有の問題が発生したときに対処できない

「コマンドを知っていることと、一連の作業を経験して判断を積み重ねていることはちがうからね。推論能力の高いモデルならうまく対処できるケースはあるかもしれないけど、それに頼るのは運頼みで再現性の低いやり方になってしまう。だから Jujutsu 固有のメンタルモデルに沿ったワークフローを AI に教えてあげる手順が必要なの」

「なるほど」

「その手順について、Claude Code では **Rules** 中心で運用するのがいいと思う⁵⁴。`CLAUDE.md` にはバージョン管理に Jujutsu を使うことだけを書いておく。そして `rules` ファイルのほうに具体的なワークフローの手順を記述する」

「`CLAUDE.md` にまとめて書くのじゃダメなんですか？」

「実のところ Claude Code では、`CLAUDE.md` と `rules` ファイルは読み込まれるタイミングも同じで⁵⁵、どちらに指示を書こうと優先度も同じなのね。`rules` に分割する理由は、純粹に管理のしやすさ。`CLAUDE.md` には、他にプロジェクト固有の情報や指示なども記述するわけだから、そこに分量の大きなバージョン管理の作

⁵⁴ 「Claude があなたのプロジェクトを記憶する方法 - Claude Code Docs」
<https://code.claude.com/docs/ja/memory>

⁵⁵ `paths` frontmatter がないときの挙動。`rules` ファイルの frontmatter に `paths` フィールドを設定すると、そのパターンに一致するファイルで作業するときのみ読み込まれ、適用される。

法を併記するのは混乱の元になる。切り出しておけば更新もレビューもしやすいし、同じルールセットを他のリポジトリにも使い回しやすい。そして個別のコマンドの禁止については、**Permissions 設定**で担保する。これについては後でくわしく説明することしよう」

「ふむふむ、rules で運用するのは管理がしやすいからと。ちなみに **Skills** を使うというのはどうでしょう？ `skills.sh`⁵⁶で『jujutsu』を検索すると、けっこういろんな skills がヒットします。これだけあるってことは、それなりに使われてるように思えるんですが」

「うーん、あまりうまくいかないんじゃないかな。skills は AI エージェントが『これは関連がある』と判断したときに読みに行くリファレンスであって、常時適用される行動制約じゃないからね。多くの場合は明示的に指定しないと使ってくれない。具体的に『push して』とリポジトリ操作を頼むような場合ならともかく、ただ機能実装やデバッグを依頼したときは skills の参照がトリガーされないまま `git` コマンドが実行されてしまう恐れがある」

「そうなんですな」

「では、その rules のファイルには何を書いたらいいか。私はおおむね次のようなことを記述してる」

1. 各種コマンドの制約
2. 差分を表示するコマンドに `--git` オプションを付けるなどして AI が理解しやすくする
3. 参照系コマンドに適宜 `--ignore-working-copy` を付けることで、revision の細分化と working copy の stale エラーを防ぐ
4. タスク開始時における change の取り扱い方
5. change の分割や conflict の解消手順
6. push 時の bookmark の適切な運用法
7. working copy の stale エラー発生時の対処法

⁵⁶ Vercel が 2026 年 1 月に公開した、AI エージェント用の skills を検索・管理できるオープンなディレクトリ兼ランキングサイト。
<https://skills.sh/>

「これらは私が実際に AI エージェントに Jujutsu を使わせる中で生じたトラブルを予防するためのものに加え、想定されるケースをひとつひとつ相談して AI エージェント自身が必要だと申告したもののリストになってる。数カ月これで運用してるけど、GitHub と連携した通常の開発業務なら問題なく機能するはず」

「2 の `--git` オプションを付けるというのは、AI にとってはよく訓練されてる Git 形式の差分のほうが発読しやすいからですよ。巷に出回ってる Jujutsu 用の skills には、こういった視点での配慮はあまりなさそう。次の 3 の `--ignore-working-copy` というのは何ですか？」

「`working copy` と履歴に差分がある状態で `jj` コマンドを実行すると、通常はスナップショットがとられるよね。その挙動を無効にするのがこのオプション。目的のひとつは、ファイルを変更するたびにスナップショットが取られて `evolution log` が必要以上に細かく分割されるのを防ぐため」

「もうひとつは？」

「AI が作業を終えるのを待っているとき、自分は別のペインで他の作業をすることがよくあるよね。もし AI による `jj` コマンドの実行と自分によるそのタイミングが重なった場合、一方のコマンドの実行が終わらないうちにもう一方がスナップショットをとって履歴を進めてしまうことがある。するとそのプロセスから見て `working-copy commit` が履歴の示す `working-copy commit` より古くなってしまい、その不整合からエラーが発生する。そういった事態をできるだけ予防するためだね。また、それが発生してしまったときの対処法も 7 で提示してる」

「ふーむ、そんな配慮も必要なんですね」

「それから、Codex CLI でのやり方についても説明しておこうか。Codex には Claude Code の rules に該当する機能がない。同じ名前でも rules という機能があるけど、こっちはサンドボックス外で実行できるコマンドを管理するためのものであって、自然言語で AI エージェントに行動規範を与えるものじゃないの」

「困りましたね。じゃあどうするんですか？」

「`AGENTS.md` と skills の併用で乗り切るのが次善策かな。ただしさっきも話したように、skills は関連があると AI が判断しないかぎり使ってくれない。だからこちらの場合は `AGENTS.md` の記述が多めになる」

「うーむ、さじ加減が難しいんですね。こんなに AI に合わせてアプローチの方法

3-1. AI エージェントを Jujutsu と協調させる

を変えないといけないとは」

「さてここまでの話を総合すると、プロジェクトごとの AI エージェント用の設定ファイルはこんなふうに置くことになる⁵⁷」

リスト 1: エージェント用設定ファイルの配置

```
my-project/
  CLAUDE.md          ← Jujutsuを使うことを指示
  AGENTS.md          ← 基本的な挙動とskillsの発動条件
  .claude/
    rules/
      jujutsu-rules.md ← 該当のrules ファイル
  .agents/
    skills/
      jujutsu/
        SKILL.md      ← Codex向けに詳細をskill化したもの
```

リスト 2: jujutsu-rules.md の冒頭部分

```
# Jujutsu (jj) Rules for AI Agents

jjの標準的なセマンティクスはAI エージェントにとって既知のものと想定する。ここに記述するのはそこから推察しえない、Jujutsu ネイティブのワークフロー、およびリポジトリ固有の設定やエイリアスを前提とした手順である。

---

## AI エージェントへの重要な注意

### コマンドの制約

以下は `settings.json` で明示的に禁止されている。これらを前提にしたプランを立ててはならない。どうしても必要な場合は、コマンドを提示してユーザー自身に実行してもらうこと。

| 禁止コマンド | 理由 |
| --- | --- |
| `git` (全般) | jjの状態を壊すおそれがある。`jj git ...` と `gh` は引き続き使用可。 |
| `jj resolve` | マージエディタが開く。conflictはファイルを直接編集して解消する。
```

⁵⁷ <https://github.com/klemiwary/Juju-chu-ja/tree/main/config/ch3>

```
|
| `jj diffedit` | diff エディタが開く。
|
| `jj arrange` | 対話的なTUI。
|
```

`jj split` は使用できるが、**必ず`fileset`と`-m`を渡すこと。引数なしでの実行、および`-i`と`--tool`の指定は、diff エディタが起動してしまうためいずれも禁止。**

また次のコマンドは実行のたびにユーザーへの確認が必要になる。作業の途中で打つではなく、タスクの終盤にまとめること。

```
`jj git push`, `jj bookmark delete`, `jj bookmark forget`, `jj bookmark track`, `jj bookmark untrack`
```

jjでワークフローがどう変わるか

- **stagingもstashも「未保存の変更」も存在しない。** ファイルを保存した瞬間、その内容はworking-copy commit(`@`)の一部になる。「この変更を含めますか?」とユーザーに尋ねてはならない。すでに含まれている。

:

「既存のリポジトリはGitで管理してるはずだから、ユーザー設定ではなくJujutsuを使っているリポジトリごとに設定を置くのがいいだろうね」

「いいですね！ これそのまま、いただいちゃいます！」

3-1-2. jj コマンドの Permissions 設定

「Claude Code の **Permissions 設定**⁵⁸は、任意のコマンドの実行を **allow**（自動許可）/ **ask**（都度確認）/ **deny**（拒否）の3種類のルールに従わせることができるというもの。先のrulesには『Gitコマンドは使用禁止』といったことを書いたけど、このpermissions設定なら確実に禁止できる。また他のjjコマンドそれぞれについても、いちいち対話で許可しなくても実行していいものを列挙しておけば、開発がスムーズに進められるよね」

⁵⁸ 「権限を設定する - Claude Code Docs」
<https://code.claude.com/docs/ja/permissions>

著者紹介

大岡由佳（おおおか ゆか）

国内の IT 企業数社での PHP や Ruby による開発およびプロダクトマネージャを経て、2016 年からフリーランスに転身。当時まだ日本ではマイナーな技術だった React に注目し、React 専門のフリーランスエンジニアとして複数の現場を渡り歩く。その経験を元に上梓した「りあクト！」シリーズが評判を呼び、累計部数 4 万部のヒット作となる。

X アカウントは @oukayuka。Jujutsu を使いこなすためのブログを juju-chu.com/ja/blog で公開中。



著者のデスクトップ環境

イラスト：黒木めぐみ（くろき めぐみ）

漫画家、イラストレーター。

「りあクト!」「じゅじゅちゅ!」シリーズの表紙イラストを一貫して担当。

じゅじゅちゅ！

jj new で始める Jujutsu x AI ワークフロー

2026 年 4 月 10 日 第 1 刷発行
2026 年 9 月 6 日 電子版バージョン 2.1.0

著 者 大岡 由佳
連 絡 先 oukayuka@gmail.com
発 行 元 くるみ割り書房
https://klemiwary.com

本書の一部、または全部を無断で複写、複製、転載、データ配信することを禁じます。

© 2026 Yuka Ooka / Klemiwary Books